

PART VI — THE SELF-LEARNING LAYER

Chapter 36B

Feature-Flag-Driven Development for RAG Pipelines

“A pipeline stage that cannot be turned off cannot be isolated, and a stage that cannot be isolated cannot be debugged.”

Chapter 36B — Feature-Flag-Driven Development for RAG Pipelines

Every mechanism this book covers — NAC, DC, LBC, retrieval validation, dedup-merge, answer drafting, auto-distillation — exists in this project's code behind its own `'ENABLE_*` boolean in `'config.py'`. This chapter steps back from any single stage to look at the flag architecture itself: why every stage needed its own kill-switch, how this project used flag combinations as an actual debugging methodology rather than only a deployment convenience, and how a per-request override system lets a single running service serve different configurations to different callers simultaneously.

The chapter draws directly on a real bug this project's own ledger records — BUG-022, found specifically **because** flag-combination testing existed — as the clearest evidence that a feature-flag architecture is not free: every flag doubles the number of code paths that must actually work, and a codebase that adds flags without testing the states they create is accumulating a debugging burden it has not yet paid for.

36B.1 Why every pipeline stage needs a kill-switch

Definition — Feature flag (this project's usage)

A boolean configuration constant that fully enables or disables one pipeline stage's execution, allowing that stage to be isolated for debugging, A/B compared against its absence, or disabled entirely in a degraded-but-functional deployment.

A pipeline with no flags at all is a pipeline where every bug report starts from the full, entangled system — there is no way to ask "does this problem still happen with compression turned off" without editing code and redeploying. Chapter 34's merge-similarity logic, Chapter 22B's three-stage compression, Chapter 16's draft-then-judge cycle: each is complex enough on its own that isolating it from its neighbors is often the fastest way to determine whether an observed problem originates inside that stage or somewhere else entirely.

This is precisely the isolation Research topic 25 in this project's own research ledger describes `'run_combinations.py'` providing: "running identical queries through different flag combinations exposed hidden dependencies, undocumented assumptions, and unsupported execution paths that normal testing never exercised." A flag is not merely an off switch for production — it is a debugging instrument, deliberately built to let a specific stage's contribution be isolated from everything around it.

36B.2 The config.py flag catalogue

This project's flags cluster into recognizable functional areas rather than forming one undifferentiated list: sub-query generation, retrieval dedup/merge (with its own nested validation flag), retrieval validation, the three independent compression stages (NAC, DC, LBC) plus their shared validation flag, answer draft creation, answer quality checking, automatic distillation, QA-pair generation, and — cutting across all of the above — a single global LLM output-repair switch.

Twelve independent kill-switches, one per pipeline stage

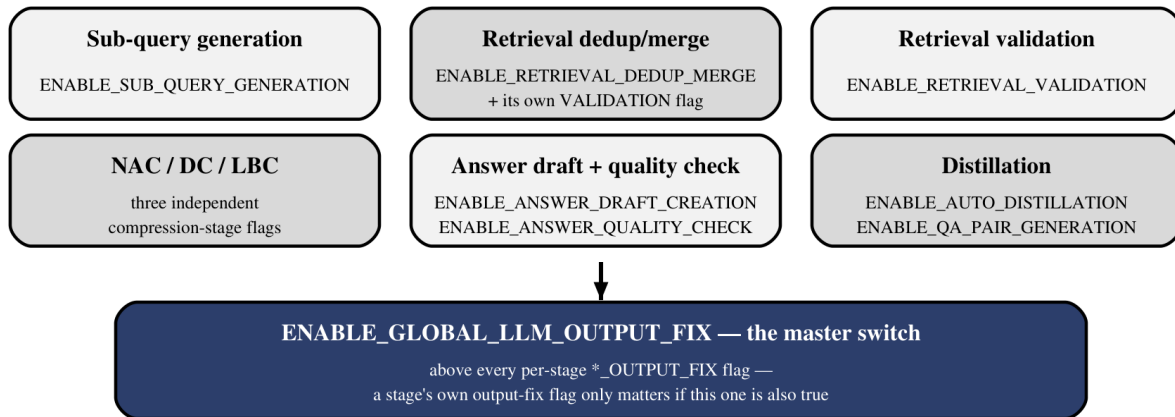


Figure 36B.1 — Flags cluster by functional area; the global output-fix switch sits above every per-stage flag as a second, independent axis of control.

Grouping by functional area rather than reading the flags as one flat list matters for exactly the reason Chapter 36B.8 formalizes later: a flag whose neighbors are all part of the same stage (NAC's `'ENABLE_NAC_COMPRESSION'` next to DC's and LBC's own flags) invites the natural question of whether all three should default together or whether independent control is genuinely needed — a question a flat, alphabetized list of twenty booleans makes much harder to ask.

36B.3 Per-stage output-fix flags as an independent layer

`'ENABLE_QA_PAIR_OUTPUT_FIX'`, `'ENABLE_RETRIEVAL_VALIDATION_OUTPUT_FIX'`, `'ENABLE_COMPRESSION_OUTPUT_FIX'`, and their siblings are not duplicates of their parent stage's own flag — they control a genuinely separate concern, Chapter 13B's multi-tier JSON repair pipeline, layered on top of whether the stage itself runs at all. A stage can be enabled with its own output-fix flag disabled, meaning that stage runs but any malformed LLM output it produces falls back to `'_parse_to_python()'` directly, with no repair attempt.

```
if ENABLE_QA_PAIR_OUTPUT_FIX and ENABLE_GLOBAL_LLM_OUTPUT_FIX:
    llm_result, _ok = fix_llm_output("distill_qa", raw, llm=self.llm)
else:
    llm_result = _parse_to_python(raw)
```

Both conditions must hold — the stage-specific flag `*and*` `'ENABLE_GLOBAL_LLM_OUTPUT_FIX'` — which is precisely the two-axis design Figure 36B.1's stacked master switch depicts: the global flag is a single, fast way to disable repair everywhere at once (useful for isolating whether a bug lives in the repair layer itself), while the per-stage flags allow finer-grained control once the global switch is confirmed on.

BUG-022 is the direct cost of this flexibility, discovered specifically through the `'run_combinations.py'` ladder testing Section 36B.1 already named: several fallback paths in `'validators.py'` and

``context_compression.py`` were written assuming output repair had already run, and crashed with ``AttributeError`` when a flag combination disabled it. The fix — routing the no-repair fallback through ``_parse_to_python()`` explicitly rather than assuming pre-parsed structure — is the general lesson every new flag combination should be validated against: a flag that is never actually tested in its "off" state is a flag whose off state is unverified, not one that is known to work.

36B.4 Building an all-flags-true baseline run

Flag-combination testing needs a fixed reference point the same way Chapter 36.1's evaluation needed a baseline: one run with every flag at its default ``True`` value, against the identical fixed question set Chapter 36.2 already catalogued, captured and archived before any flag gets deliberately flipped off. Every subsequent single-flag-disabled run is compared against this one reference, isolating the effect of exactly one changed variable at a time.

This baseline-first discipline is the same principle Chapter 36.1 applied to self-learning evaluation, generalized to configuration testing broadly: a difference is only interpretable relative to a known, fixed starting point. A flag-combination test run with no baseline to compare against can observe that a flag-off run **produces** some output, but cannot say whether that output represents a regression, an improvement, or simply a different valid path through the pipeline.

36B.5 Disabling one subsystem at a time

The "all flags true except one" methodology this project's own ``run_combinations.py`` implements is a ladder in the literal sense: start from the all-true baseline, flip exactly one flag to ``False``, run the identical fixed question set, and diff the result against the baseline before moving to the next flag. This isolates each stage's individual contribution to the pipeline's behavior — a change observed with only ``ENABLE_NAC_COMPRESSION=False`` is attributable specifically to NAC, uncontaminated by any other simultaneous change.

Common pitfall — Disabling multiple flags at once to save time

Flipping two flags off simultaneously to test faster halves the number of runs needed but destroys the isolation the whole methodology exists to provide — an observed difference could now be caused by either flag, by their interaction, or by neither alone. One flag per run is slower but is the only version of this test that actually answers "what does this specific stage do."

36B.6 The post-retrieval-separation dry run

The single-flag-at-a-time ladder is not the only use for this methodology — comparing chunk output **before** and **after** a structural refactor is the identical discipline applied to an architecture change rather than a flag. When retrieval was split from a single merged pool into the two-track parallel design Chapter 33.6 covers, the validating question was not "does a flag work" but "do the two collections still retrieve the same underlying content they did before the split, just organized differently" — answered by running the identical fixed question set through both the pre-split and post-split code and diffing the resulting chunk sets question by question.

This is a structural application of Chapter 36.4's comparison discipline: any change large enough to alter what a pipeline retrieves or produces — a flag flip or a full architectural refactor — earns the identical treatment, a fixed question set run before and after, compared at a granularity fine enough to catch a real difference hiding inside an unchanged aggregate.

36B.7 Cross-run diffing for regression hunting

This project's own `extract_logs.py` (Research topic 40, and Chapter 36C's own subject) is the general-purpose tool this comparison methodology depends on at scale: it parses debug logs from any two runs and extracts the specific, comparable fields — retrieval parameters, per-chunk scores, validation verdicts, LLM call counts — needed to diff them meaningfully, rather than requiring a human to manually scroll through two multi-thousand-line raw logs looking for differences by eye.

A structured diff tool matters here for the identical reason Chapter 24.6's `[CTXSIZE]` instrumentation mattered for context-size stress testing: the raw capability to run two configurations and eyeball the difference exists without any special tooling, but at any real scale — fifteen batches, a hundred questions, twenty flags — manual comparison stops being practical long before it stops being theoretically possible. The tooling is what makes the methodology repeatable rather than a one-time, heroic manual effort.

36B.8 What to flag-gate vs. what to hard-wire

Not every piece of configurable behavior deserves a flag, and BUG-022 is the concrete argument for restraint: each flag is a real, ongoing maintenance cost, since every fallback path it creates must be independently tested and kept correct. A reasonable heuristic, consistent with which constants this project actually gated versus left hard-coded: flag-gate a stage if disabling it produces a legitimately different but still-functional pipeline (skipping LBC still produces a usable, if less refined, answer); hard-wire a constant if there is no meaningful "disabled" behavior for it to fall back to.

Question to ask	Flag-gate if...	Hard-wire if...
Does a "disabled" state make sense?	Yes — the pipeline still functions without it	No — there is no coherent behavior with it "off"
Will it need A/B comparison?	Yes — Chapter 36.1's evaluation discipline applies	No — its value doesn't change the pipeline's shape
Does disabling it require new fallback code?	The fallback is worth the maintenance cost	The fallback cost isn't justified by the flag's value

`'MERGE_SIMILARITY_THRESHOLD'` (Chapter 34.2) is a useful contrast case: it is a tunable **value**, not a stage that can be meaningfully "disabled" — there is no coherent "off" state for a similarity threshold the way there is for NAC or LBC. Constants like this belong in `'config.py'` as tunable values, revisited through Chapter 36C's evidence-based tuning methodology, but they do not need the full `'ENABLE_*` flag treatment this chapter otherwise recommends.

`'MAX_ITERATIONS'` and `'MAX_TOTAL_RETRIEVALS'` (Chapter 23) sit at a similar point on this same spectrum — genuinely tunable, but with no meaningful "disabled" state a boolean flag could express. Setting either to an arbitrarily large number would functionally disable the guardrail, but that is a very different interface from an explicit `'ENABLE_ITERATION_CAP'` flag, and this project's own choice to leave them as plain integer constants rather than flag-gating them is consistent with the heuristic this section proposes: a value with no coherent "off" state belongs in `'config.py'` as a number to tune, not in the `'ENABLE_*` catalogue as a switch to flip.

36B.9 Per-request flag overrides

`switches.py` extends the flag architecture from a single, process-wide configuration into a per-request one: `resolve_switches(overrides)` takes a request's JSON body, overlays only its known, boolean-typed keys onto `DEFAULT_SWITCHES`, and the merged dictionary is stored once in `GraphState["switches"]` for that request's entire graph execution. Every node reads the resolved value via `get_switches(state)` rather than importing the `ENABLE_*` constants directly — the identical single-source-of-truth discipline Chapter 27.6 established, applied here to per-request configuration instead of per-run counters.

Figure 36B.2 traces this resolution as a strict one-way pipeline — overrides flow in once, at the very start of a request, and every node downstream reads the identical already-resolved dictionary rather than re-consulting `config.py` or the original request body at any later point.

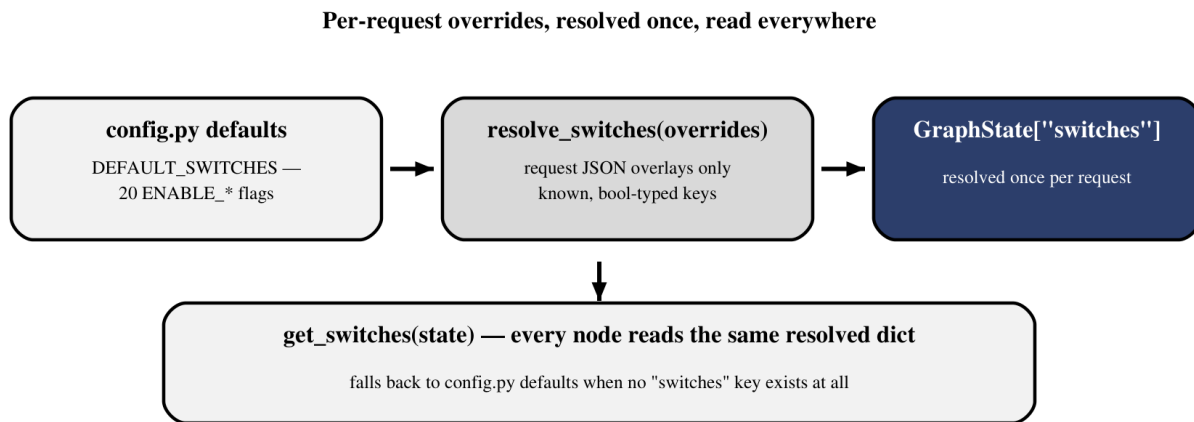


Figure 36B.2 — Overrides are resolved exactly once per request and threaded through `GraphState`; no node ever reads a raw config constant directly.

```
def resolve_switches(overrides: dict | None) -> dict[str, bool]:
    merged = dict(DEFAULT_SWITCHES)
    if overrides:
        for key, value in overrides.items():
            if key in SWITCH_NAMES and isinstance(value, bool):
                merged[key] = value
    return merged
```

The type and key-name filtering here is a deliberate hard filter (Chapter 27.4's principle, applied to a request body instead of a query string): an unrecognized key or a non-boolean value is silently dropped rather than corrupting the merged dictionary or raising an exception that would crash an otherwise-valid request over one malformed override field. This lets one running `app_workflow/api.py` instance serve a full-featured request and a deliberately degraded, minimal-pipeline request side by side, from the identical deployed code, differing only in what each request's own body asked to override.

The `get_switches()` fallback deserves its own note, because it is what keeps this per-request system backward-compatible with entry points that never adopted it. `main.py`'s CLI entrypoint calls `run_agent()` without ever constructing a `GraphState["switches"]` entry at all, and `get_switches(state)` handles that absence gracefully: `state.get("switches")` or `DEFAULT_SWITCHES` falls through to the process-wide defaults exactly as if no override system existed. A node written against `get_switches()` behaves identically whether it is called from a request that specified overrides, a request that specified none, or a caller that never knew the override system existed at all — one function signature, three call patterns, no special-casing required in the node's own logic.

Chapter 36C closes Part VI by applying this exact evidence-based discipline — baseline, controlled comparison, structured log diffing — to the specific numeric thresholds Chapter 33 introduced, turning `'RETRIEVAL_TOP_K'`, `'DOCUMENTS_MIN_SIMILARITY'`, and their siblings from reasoned defaults into values confirmed, or corrected, by this project's own production logs.

The thread connecting every section of this chapter is worth naming explicitly as it closes: a flag is only as valuable as the discipline built around using it. `'ENABLE_NAC_COMPRESSION'` sitting in `'config.py'`, unused by any deliberate testing methodology, is barely more useful than no flag at all — its value comes entirely from Section 36B.5's ladder testing actually exercising its off state, from Section 36B.9's per-request overrides actually letting a caller choose it deliberately, and from BUG-022's lesson actually being absorbed into how new flags get validated going forward. A feature-flag architecture is infrastructure for a testing discipline, not a substitute for one.